

GPU Governor Energy-Efficiency Analysis

30 CUDA workloads × 4 GPU governors × 10 repeats

separately under CPU 3354 MHz and CPU powersave 858 MHz

Headline

Use GPU ondemand as the general energy-aware policy.

Reserve GPU powersave for validated low-utilization workloads.

GPU performance has no replicated energy benefit.

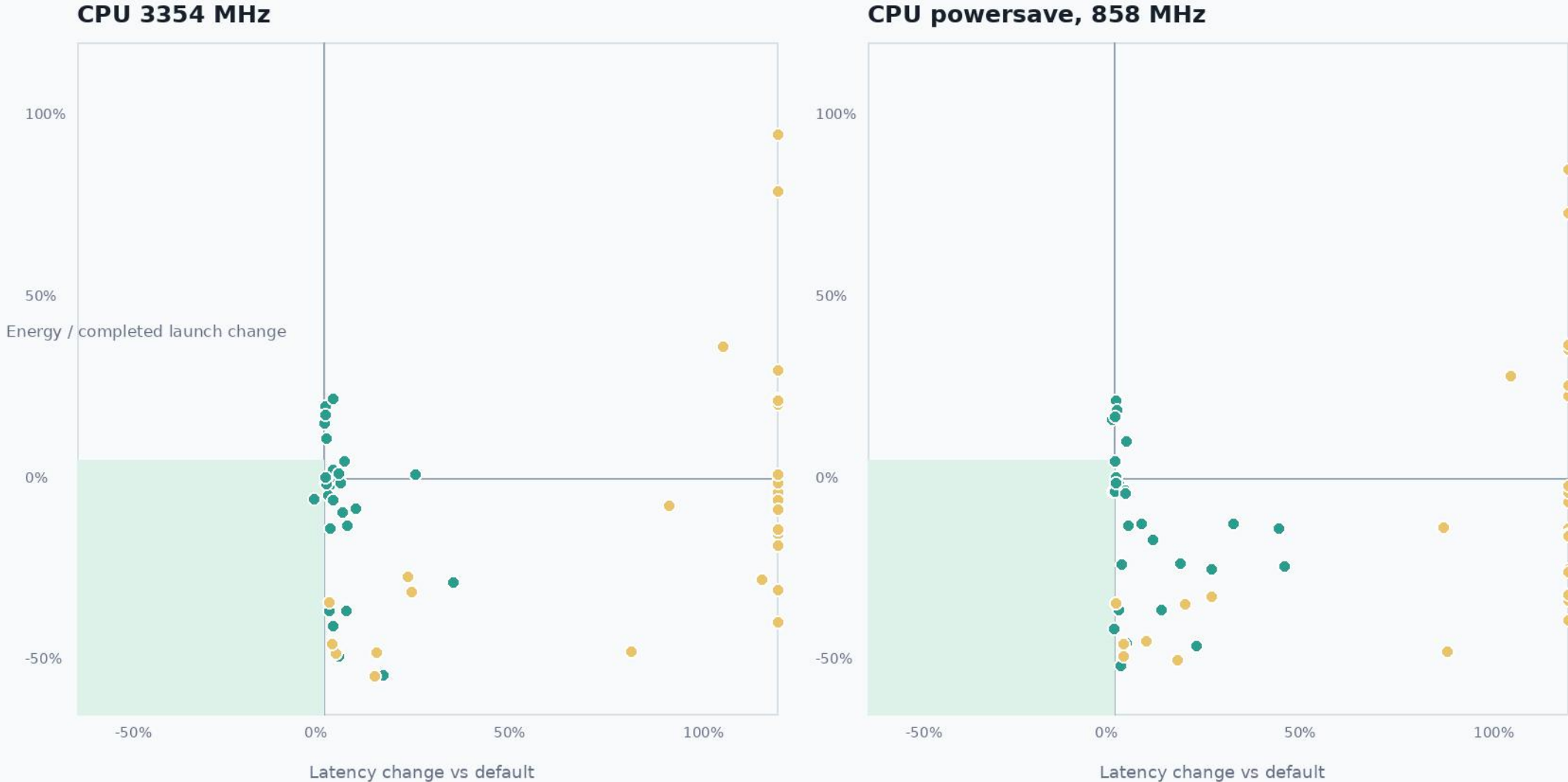
Executive findings

- GPU ondemand is the safest broad alternative to NVIDIA default. Median energy per completed launch changes were -1.6% at CPU 3354 MHz and -7.9% at CPU 858 MHz, with typical latency costs around 2-3%.
- For low-SM-activity workloads (<30% under GPU default), ondemand reduced median energy by 37-38% for only 2-3% median latency cost.
- GPU powersave cut average power by about 73%, but median latency increased 129-137%. It is not a universal efficiency policy.
- The strongest near-free powersave wins replicated in both CPU modes: backprop, bfs, and hybridsort. Each saved roughly 34-50% energy with $\leq 5\%$ latency cost.
- GPU performance raised median energy 12-14% and produced no benchmark with replicated energy improvement across both CPU modes.
- The apparent advantage of CPU 858 MHz is provisional: CPU modes were measured a week apart, with different memory/swap states, so the comparison is confounded.

Energy vs. latency trade-off

Each point is one benchmark relative to GPU default; medians use 10 repeated runs

ondemand powersave



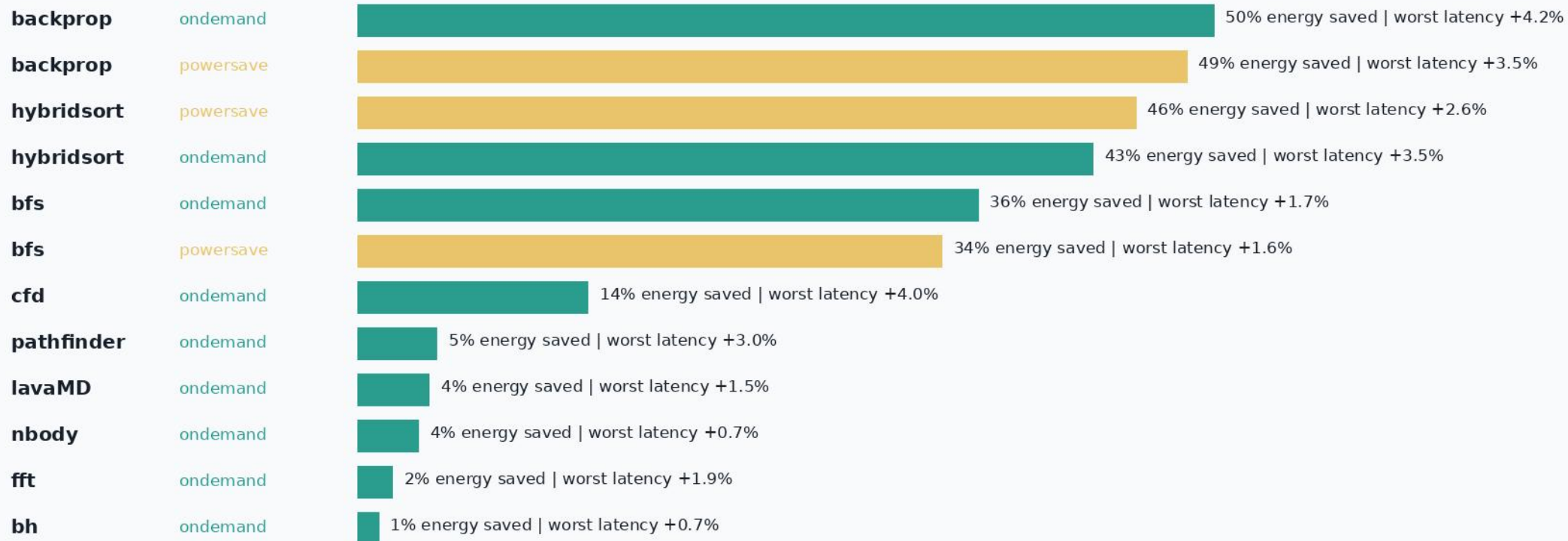
Workload-specific energy effects

Percent energy per completed launch vs GPU default; gray cells failed to complete work

	CPU 3354 MHz ondemand	CPU 3354 MHz powersave	CPU 3354 MHz performance	CPU 858 MHz ondemand	CPU 858 MHz powersave	CPU 858 MHz performance
babelstream	+20%	-31%	+17%	+21%	-33%	+20%
backprop	-49%	-48%	+53%	-51%	-49%	+55%
bfs	-37%	-34%	+96%	-36%	-34%	+111%
bh	-1%	+95%	-3%	-1%	+85%	+2%
cfd	-14%	-31%	+5%	-13%	-33%	+11%
fft	-2%	+79%	-1%	-2%	+73%	+3%
gaussian	-13%	-27%	+103%	-24%	-35%	+71%
gemmEx	-1%	FAIL	+0%	+0%	FAIL	+2%
heartwall	-8%	+22%	+3%	-12%	+23%	+5%
hotspot	+0%	-15%	+1%	-3%	-25%	+3%
hybridsort	-41%	-46%	+74%	-45%	-46%	+72%
ising	+11%	-4%	+14%	+5%	-14%	+10%
kmeans	+2%	-18%	+7%	-25%	-15%	+11%
lavaMD	-5%	+36%	+55%	-4%	+28%	+60%
leukocyte	-29%	+20%	+10%	-24%	+26%	+6%
lud	+15%	-48%	+17%	+19%	-48%	+20%
lulesh	-1%	+30%	+9%	-24%	+35%	+15%
memcpy	+1%	-8%	+46%	-12%	-14%	+27%
mixbench	+0%	FAIL	+1%	-4%	FAIL	+2%
myocyte	-2%	FAIL	-5%	-14%	FAIL	+1%
nbody	-6%	-1%	-4%	-1%	-6%	+0%
nn	+1%	+1%	+4%	-36%	-32%	+2%
nw	-37%	-48%	+34%	-42%	-45%	+51%
particlefilter	+0%	+21%	-4%	+16%	+37%	+17%
pathfinder	-6%	-6%	+52%	-3%	-4%	+65%
srad	+22%	-40%	+13%	+10%	-39%	+22%
stencil1d	+17%	-14%	+20%	+17%	-16%	+22%
stencil3d	-9%	-8%	+21%	-4%	-2%	+29%
streamcluster	-54%	-55%	+23%	-46%	-50%	+36%
triad	+5%	-28%	+12%	-17%	-26%	+12%

Low-hanging opportunities

Only policies replicated in both CPU modes with energy savings and $\leq 5\%$ latency cost



Counter interpretation and policy

Counters that matter

- Within-workload normalized SM and video clocks are the strongest power-state indicators: Spearman rho ≈ 0.91 – 0.94 with average power and ≈ 0.57 – 0.64 with energy per completed launch.
- GPU and CPU temperatures track power, but they are lagging thermal outcomes and run-order indicators, not independent control variables.
- SM-active percentage becomes negatively associated with power after governor changes because low clocks prolong kernels inside the fixed 120-second window. Do not interpret this as activity saving power.
- CPU utilization and page-fault counters are weaker and unstable after within-workload normalization. They should not drive governor selection from this dataset.

Conservative runtime policy

- Unknown workload or strict latency: GPU default.
- Default SM active $< 30\%$: trial GPU ondemand; promote if completed work per joule improves and latency stays inside budget.
- Use GPU powersave automatically only for a proven allow-list, starting with backprop, bfs, and hybridsort.
- Never rank a run with completed_launches=0 as efficient. gemmEx, mixbench, and myocyte failed all powersave runs in both CPU modes.

Method and limitations

- Primary metric: average platform power $\times$ actual observation duration / completed launches. This measures joules per completed unit of work and respects the fixed ~ 120 -second run design.
- Each benchmark-governor cell uses the median of 10 runs. Governor effects are percentages against the same benchmark's GPU-default median in the same CPU mode. Unlike the previous report, unlike workloads are never averaged in raw seconds or joules.
- Both files contain 1,200 runs: 30 benchmarks $\times$ 4 GPU governors $\times$ 10 repeats. The first file is fixed at 3354 MHz; the second is explicitly `cpu_powersave` and fixed at 858 MHz.
- Governor blocks were collected chronologically rather than randomized. Thermal drift, background activity, cache state, and benchmark order can therefore bias effects.
- CPU modes were collected on different dates. Early runs differ substantially in memory and swap state; CPU-frequency causality cannot be claimed.
- Follow-up: interleave CPU/GPU policies within each benchmark, randomize order, stabilize package temperature, record idle baseline, and measure direct rail joules.